

Four-Layer Architecture

Sources —► (ERP, Salesforce, EDW, IoT)

Staging —►

Raw Vault —► (Hub/Link/Sat)

Business Vault —► (PIT, Bridge/Computed sat)

Information Delivery —► (Star Schema / Vault marts)

Consumption

Unity Catalog + Workflow: Governance, security, orchestration (all layers)

Layer	Delta Schema	Purpose	Load patterns
staging	'Stg'	1:1 landing of source extracts/CDC, typed, hashed keys pre-computed	Truncate-and-load or append (Auto Loader)
Raw Vault	'raw_vault'	Hubs, Links, Satellites — pure, unmodified facts as delivered by source	Insert-only, idempotent 'MERGE'/'APPLY CHANGES'
Business Vault	'biz_vault'	PIT tables, Bridge tables, computed/derived satellites, same-as links	Scheduled rebuild or incremental 'MERGE'
Information Delivery	'gold'	Star schemas, flattened vault marts, aggregates	Views or materialized builds off Business Vault

Naming & Hash Key Conventions

Object	Prefix	Example
Hub	'hub_'	`hub_customer`
Link	'link_'	`link_customer_order`
Satellite	'sat_'	`sat_customer_details`
Multi-active Satellite	'mas_'	`mas_customer_phone`
Same-as Link	'sal_'	`sal_customer`
Hierarchical Link	'hal_'	`hal_org_unit`
Point-in-Time	'pit_'	`pit_customer`
Bridge	'brg_'	`brg_customer_order`

Hash key formula (standard across the model)

hub_hk = SHA2(business_key, 256)

link_hk = SHA2(concat_ws('||', sorted(parent_hub_business_keys)), 256)

hash_diff = SHA2(concat_ws('||', sorted(descriptive_attribute_values)), 256)

Raw Vault — DDL and Load Code – (HUB)

```
```sql
```

```
CREATE TABLE IF NOT EXISTS raw_vault.hub_customer (
 customer_hk STRING NOT NULL, -- SHA2(customer_id)
 customer_id STRING NOT NULL, -- business key, as delivered by source
 load_dts TIMESTAMP NOT NULL,
 record_source STRING NOT NULL
)

USING DELTA

CLUSTER BY (customer_hk)

TBLPROPERTIES ('delta.enableChangeDataFeed' = 'true');
```

```

```python
from pyspark.sql import functions as F

from delta.tables import DeltaTable

def load_hub(spark, staging_table, hub_table, business_key_col, record_source):

    src = (spark.table(staging_table)

        .withColumn("customer_hk", F.sha2(F.upper(F.trim(F.col(business_key_col))), 256))

        .withColumn("customer_id", F.upper(F.trim(F.col(business_key_col))))

        .withColumn("load_dts", F.current_timestamp())

        .withColumn("record_source", F.lit(record_source))

        .select("customer_hk", "customer_id", "load_dts", "record_source")

        .dropDuplicates(["customer_hk"]))

    hub = DeltaTable.forName(spark, hub_table)

    (hub.alias("t")

     .merge(src.alias("s"), "t.customer_hk = s.customer_hk")

     .whenNotMatchedInsertAll() # Hubs are insert-only — never update an existing hash key

     .execute())

load_hub(spark, "stg.customer_extract", "raw_vault.hub_customer", "customer_id", "EDW_CUSTOMER")

```

Raw Vault — DDL and Load Code – ([Link](#))

```

```sql
CREATE TABLE IF NOT EXISTS raw_vault.link_customer_order (
 link_customer_order_hk STRING NOT NULL, -- SHA2(customer_hk || order_hk)
 customer_hk STRING NOT NULL,
 order_hk STRING NOT NULL,
 load_dts TIMESTAMP NOT NULL,
 record_source STRING NOT NULL

```

```

)

USING DELTA

CLUSTER BY (link_customer_order_hk);

def load_link(spark, staging_table, link_table, parent_hk_cols, link_hk_name, record_source):

 src = spark.table(staging_table)

 concat_expr = F.concat_ws("|", *[F.col(c) for c in parent_hk_cols])

 src = (src.withColumn(link_hk_name, F.sha2(concat_expr, 256))

 .withColumn("load_dts", F.current_timestamp())

 .withColumn("record_source", F.lit(record_source))

 .dropDuplicates([link_hk_name]))

 link = DeltaTable.forName(spark, link_table)

 (link.alias("t")

 .merge(src.alias("s"), f"t.{link_hk_name} = s.{link_hk_name}")

 .whenNotMatchedInsertAll() # Links are also insert-only

 .execute())

load_link(spark, "stg.customer_order_extract", "raw_vault.link_customer_order",

 ["customer_hk", "order_hk"], "link_customer_order_hk", "EDW_ORDER")

```

## Raw Vault — DDL and Load Code – ( Satellite with hashdiff-based history)

```

```sql

CREATE TABLE IF NOT EXISTS raw_vault.sat_customer_details (

    customer_hk    STRING NOT NULL,

    load_dts       TIMESTAMP NOT NULL,

    load_end_dts   TIMESTAMP,      -- NULL = current version

    hash_diff      STRING NOT NULL,

    customer_name  STRING,

    segment        STRING,

    region         STRING,

```

```
email      STRING,  
record_source STRING NOT NULL  
)
```

```
USING DELTA
```

```
CLUSTER BY (customer_hk);
```

```
...
```

```
```python
```

```
from pyspark.sql import Window
```

```
def load_satellite(spark, staging_table, sat_table, hub_key_col, attr_cols, record_source):
```

```
 src = spark.table(staging_table)
```

```
 concat_expr = F.concat_ws("|", *[F.coalesce(F.col(c).cast("string"), F.lit("")) for c in attr_cols])
```

```
 staged = (src.withColumn("hash_diff", F.sha2(concat_expr, 256))
```

```
 .withColumn("load_dts", F.current_timestamp())
```

```
 .withColumn("record_source", F.lit(record_source)))
```

```
 # latest version per hub key currently open (load_end_dts IS NULL)
```

```
 current = (spark.table(sat_table)
```

```
 .filter("load_end_dts IS NULL")
```

```
 .select(hub_key_col, F.col("hash_diff").alias("current_hash_diff")))
```

```
 # rows that are new hub keys OR whose content changed
```

```
 changed = (staged.alias("s")
```

```
 .join(current.alias("c"), hub_key_col, "left")
```

```
 .filter("c.current_hash_diff IS NULL OR c.current_hash_diff <> s.hash_diff")
```

```
 .select("s.*"))
```

```
 if changed.head(1):
```

```
 sat = DeltaTable.forName(spark, sat_table)
```

```
 # close out the previous open row for any hub key being updated
```

```
 (sat.alias("t")
```

```

.merge(changed.select(hub_key_col).distinct().alias("s"),
 f"t.{hub_key_col} = s.{hub_key_col} AND t.load_end_dts IS NULL")
.whenMatchedUpdate(set={"load_end_dts": "current_timestamp()})
.execute()

insert new current version

(changed.withColumn("load_end_dts", F.lit(None).cast("timestamp"))
 .select(*[c.name for c in spark.table(sat_table).schema])
 .write.format("delta").mode("append").saveAsTable(sat_table))

load_satellite(spark, "stg.customer_extract", "raw_vault.sat_customer_details",
 "customer_hk", ["customer_name", "segment", "region", "email"], "EDW_CUSTOMER")

```

### **Multi-Active Satellite (attributes with multiple concurrent values, e.g. phone numbers)**

```

```sql
CREATE TABLE IF NOT EXISTS raw_vault.mas_customer_phone (
  customer_hk  STRING NOT NULL,
  phone_seq    INT NOT NULL,    -- differentiator: distinguishes concurrent rows for same
key
  load_dts     TIMESTAMP NOT NULL,
  load_end_dts TIMESTAMP,
  hash_diff    STRING NOT NULL,
  phone_number STRING,
  phone_type   STRING,
  record_source STRING NOT NULL
)
USING DELTA
CLUSTER BY (customer_hk, phone_seq);
```

```

## Same-As Link (master data / duplicate resolution across sources)

```
```sql
-- Used when the same real-world customer arrives from two source systems with different keys
CREATE TABLE IF NOT EXISTS raw_vault.sal_customer (
  sal_customer_hk STRING NOT NULL,
  customer_hk_1  STRING NOT NULL, -- hub key from source A
  customer_hk_2  STRING NOT NULL, -- hub key from source B (matched/master-data-resolved)
  load_dts       TIMESTAMP NOT NULL,
  record_source  STRING NOT NULL
) USING DELTA CLUSTER BY (sal_customer_hk);
```
```

## Business Vault — PIT, Bridge, and Computed Satellites

Business Vault is where Data Vault typically earns its "fast and reliable" reputation on Databricks: pre-joining Hub + Satellite + Link chains via PIT and Bridge tables is what makes downstream Gold marts fast without repeatedly walking Raw Vault joins.

### Point-in-Time (PIT) Table

A PIT table pre-computes, per Hub key and snapshot date, which Satellite row was current — eliminating expensive as-of joins at query time.

```
```python
def build_pit_customer(spark, snapshot_date):
    hub = spark.table("raw_vault.hub_customer")
    sat = spark.table("raw_vault.sat_customer_details")

    pit = (hub.alias("h")
           .join(sat.alias("s"), "customer_hk")
           .filter((F.col("s.load_dts") <= snapshot_date) &
                  (F.col("s.load_end_dts").isNull() | (F.col("s.load_end_dts") > snapshot_date))))
```
```

```

 .select("h.customer_hk", "h.customer_id",
 F.lit(snapshot_date).alias("snapshot_dts"),
 F.col("s.load_dts").alias("sat_customer_details_load_dts"))

 (pit.write.format("delta").mode("overwrite")
 .option("replaceWhere", f"snapshot_dts = '{snapshot_date}'")
 .partitionBy("snapshot_dts")
 .saveAsTable("biz_vault.pit_customer"))
...

```sql
CREATE TABLE IF NOT EXISTS biz_vault.pit_customer (
  customer_hk          STRING,
  customer_id          STRING,
  snapshot_dts         DATE,
  sat_customer_details_load_dts  TIMESTAMP  -- pointer back into the satellite, add one column per satellite
)
USING DELTA
PARTITIONED BY (snapshot_dts);

```

Bridge Table

A Bridge table pre-resolves multi-hop Link traversals (e.g. Customer → Order → Product) so Gold-layer joins are single-hop.

```

```python
def build_bridge_customer_order(spark, snapshot_date):
 link = spark.table("raw_vault.link_customer_order")
 hub_c = spark.table("raw_vault.hub_customer")
 hub_o = spark.table("raw_vault.hub_order")

 bridge = (link.join(hub_c, "customer_hk").join(hub_o, "order_hk")
 .select("link_customer_order_hk", "customer_hk", "customer_id",

```

```
"order_hk", "order_id",
```

```
F.lit(snapshot_date).alias("snapshot_dts"))
```

```
(bridge.write.format("delta").mode("overwrite")
```

```
.option("replaceWhere", f"snapshot_dts = '{snapshot_date}'))
```

```
.partitionBy("snapshot_dts")
```

```
.saveAsTable("biz_vault.brg_customer_order"))
```

### Computed / Derived Satellite (business rules layer)

```
```python
```

```
# Business Vault satellite: derived customer lifetime value, refreshed on schedule
```

```
clv = (spark.table("raw_vault.hub_customer").alias("h")
```

```
.join(spark.table("gold.fact_sales"), "customer_hk", "left")
```

```
.groupBy("customer_hk")
```

```
.agg(F.sum("net_amount").alias("lifetime_value"),
```

```
      F.count("order_id").alias("order_count"))
```

```
.withColumn("hash_diff", F.sha2(F.concat_ws(" | ", "lifetime_value", "order_count"), 256))
```

```
.withColumn("load_dts", F.current_timestamp())
```

```
.withColumn("record_source", F.lit("COMPUTED_CLV"))))
```

```
clv.write.format("delta").mode("append").saveAsTable("biz_vault.sat_customer_clv")
```

```
```
```

### Information Delivery — Flattening Vault into Consumable Marts View

```
```sql
```

```
-- Gold star-schema dimension built directly off the PIT + Satellite (fast, single-hop)
```

```
CREATE OR REPLACE VIEW gold.dim_customer AS
```

```
SELECT
```

```
  h.customer_hk      AS customer_sk,
```

```
  h.customer_id,
```

```

s.customer_name,

s.segment,

s.region,

p.snapshot_dts AS as_of_date
FROM biz_vault.pit_customer p
JOIN raw_vault.hub_customer h ON p.customer_hk = h.customer_hk
JOIN raw_vault.sat_customer_details s
  ON s.customer_hk = h.customer_hk
AND s.load_dts = p.sat_customer_details_load_dts;
...

```

For high-concurrency BI, materialize this view as a managed Delta table on a schedule instead of a live view:

```

```python
(spark.table("gold.dim_customer") # the view above
 .write.format("delta").mode("overwrite")
 .saveAsTable("gold.dim_customer_materialized"))

```

## Testing & Validation for Data Vault Migrations

1. Hash key uniqueness (no collisions)

```
assert spark.table("raw_vault.hub_customer").groupBy("customer_hk").count().filter("count > 1").count() == 0
```

2. Every Link parent key must exist in its Hub (referential integrity)

```

orphans = (spark.table("raw_vault.link_customer_order")
 .join(spark.table("raw_vault.hub_customer"), "customer_hk", "left_anti"))
assert orphans.count() == 0, "Orphaned link rows found — hub load ran out of order"

```

3. Satellite has exactly one open row (load\_end\_dts IS NULL) per hub key

```

dupes = (spark.table("raw_vault.sat_customer_details")
 .filter("load_end_dts IS NULL")
 .groupBy("customer_hk").count().filter("count > 1"))

```

```
assert dupes.count() == 0, "Multiple open satellite rows for same key"
```

### Legacy EDW → Data Vault Mapping

Legacy EDW object	Data Vault target
Dimension table (e.g. `dim_customer`)	`hub_customer` + `sat_customer_details`
Fact/transaction table	`link_*` between relevant hubs + satellite(s) on the link for measures
Bridge/junction table (many-to-many)	`link_*` directly
Slowly changing dimension history	Satellite history (`load_dts`/`load_end_dts`)
Master data / golden record table	Same-As Link (`sal_*`) over multiple source Hubs
Org hierarchy / parent-child table	Hierarchical Link (`hal_*`)
Reporting view / BI semantic layer	Information Delivery layer (Gold views/materialized marts off PIT/Bridge)

Raw Vault is the immutable, source-faithful core (Hub = key, Link = relationship, Satellite = history) — load it insert-only and idempotently.

Business Vault (PIT, Bridge, computed satellites) is where Databricks-specific performance work pays off, turning multi-hop vault joins into single-hop Gold queries.

Everything is governed uniformly through Unity Catalog and scheduled through Workflows/DLT, exactly as in the base medallion architecture — Data Vault simply defines what lives inside Silver.